COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE010	**SECTION:**	CPE21S1
DATE PERFORMED:	9/8/2026	**DATE SUBMITTED:**	9/8/2026
NAME:	Al Twain D. Pasion	**INSTRUCTOR:**	Engr Jimlord Quejado

ACTIVITY NO. 7.1 **Sorting Algorithms**

1. PROCEDURE OUTPUT

What procedures did you perform?

Table 7-1. Array of Values for Sort Algorithm Testing

CODE

```
int dataset[size];

randomArray(dataset, size);

std::cout << "\nUnsorted Dataset1: \n";
displayArray(dataset, size);

std::cout << "\nBubblesort: \n";
bubbleSort(dataset, size);
displayArray(dataset, size);
```

```
void displayArray(int arr[], int size) {
    for (int i = 0; i < size; i++) {
        std::cout << arr[i] << " ";
    }
}

void randomArray(int arr[], int size) {
    for (int i = 0; i < size; i++) {
        arr[i] = rand() % 100;
    }
}
```

OUTPUT

```
Unsorted Dataset1:
8 59 28 82 1 19 0 2 79 41 73 28 89 4 74 48 77 63 46 69 54 61 26 94 43 77 65 26 60 91 67 49 64 41 70 55 95 58 43 4 45 41
35 4 71 53 1 95 84 92 75 92 94 0 35 95 2 97 48 73 1 40 44 67 4 59 25 90 15 82 47 81 82 42 80 4 4 96 87 91 55 33 21 10 31
 19 37 15 30 30 37 48 8 20 77 99 77 37 52 59
```

OBSERVATIONS

The srand(time(0)) allows us to give a different set of values each run. The random function with modulo 100 allows us to simplify the digits given. Now we put the values in the array and print out the unsorted data set.

Table 7-2. Bubble Sort Technique

CODE

```
std::cout << "\nBubblesort: \n";
bubbleSort(dataset, size);
displayArray(dataset, size);
```

```
template <typename T>
void bubbleSort(T arr[], int size) {
    for (int i = 0; i < size - 1; i++) {
        for (int j = 0; j < size - i - 1; j++) {
            if (arr[j] > arr[j + 1]) {
                std::swap( a: [&] arr[j], b: [&] arr[j + 1]);
            }
        }
    }
}
```

OUTPUT

```
Bubblesort:
0 0 1 1 1 2 2 4 4 4 4 4 4 8 8 10 15 15 19 19 20 21 25 26 26 28 28 30 30 31 33 35 35 37 37 37 40 41 41 41 42 43 43 44 45
46 47 48 48 48 49 52 53 54 55 55 58 59 59 59 60 61 63 64 65 67 67 69 70 71 73 73 74 75 77 77 77 77 79 80 81 82 82 82 84
87 89 90 91 91 92 92 94 94 95 95 95 96 97 99
```

OBSERVATION
Bubble sorting swaps the adjacent elements if they are greater/less than the other. The time complexity of bubble sorting is O(n^2) because of the nested loops. The outer loop is for how many times we need to pass through the array, and the inner loop is responsible for swapping adjacent elements.

Table 7-3. Selection Sort Algorithm

CODE

```
int main() {
    srand( Seed: time(0));

    int dataset[size];

    randomArray(dataset, size);

    std::cout << "\nUnsorted Dataset1: \n";
    displayArray(dataset, size);

    std::cout << "\nSelectionsort: \n";
    selectionSort(dataset, size);
    displayArray(dataset, size);

    return 0;
}
```

```cpp
template <typename T>
int routine_smallest(T A[], int max_size, int k) {
    int position = k;
    T smallestElement = A[k];

    for (int j = k+1; j < max_size; j++) {
        if (A[j] < smallestElement) {
            smallestElement = A[j];
            position = j;
        }
    }
    return position;
}

template <typename T>
void selectionSort(T arr[], int arrSize) {
    int pos;
    for (int i = 0; i < arrSize-1; i++) {
        pos = routine_smallest(arr, arrSize, k: i);
        std::swap( a: [&]arr[i], b: [&]arr[pos]);
    }
}
```

OUTPUT

```
Unsorted Dataset1:
70 13 76 21 9 62 78 6 54 38 87 63 26 12 77 28 96 45 26 91 80 22 62 25 54 84 5 99 90 75 55 96 66 14 0 89 4 29 27 9 45 12
58 1 88 20 91 39 55 84 81 95 19 74 46 98 66 93 20 91 6 24 89 2 55 60 47 55 65 63 52 14 87 90 51 92 11 77 79 46 7 28 59 2
5 47 16 10 62 81 41 96 14 21 94 86 97 37 4 26 54
Selectionsort:
0 1 2 4 4 5 6 6 7 9 9 10 11 12 12 13 14 14 14 16 19 20 20 21 21 22 24 25 25 26 26 26 27 28 28 29 37 38 39 41 45 45 46 46
 47 47 51 52 54 54 54 55 55 55 55 58 59 60 62 62 62 63 63 65 66 66 70 74 75 76 77 77 78 79 80 81 81 84 84 86 87 87 88 89
 89 90 90 91 91 91 92 93 94 95 96 96 96 97 98 99
```

OBSERVATION

After generating a new set of random values, I then used the selection sort function. Selection sorting has a time complexity of O(n^2) because the routine smallest for loop is nested in the selection sort for loop. Selection sorting gets the minimum value using the routine smallest function of the unsorted region and puts it in the sorted region in the right position. This then gets repeated until the function completes the array.

Table 7-4. Insertion Sort Algorithm

CODE

```
int main() {
    srand( Seed: time(0));

    int dataset[size];

    randomArray(dataset, size);

    std::cout << "\nUnsorted Dataset1: \n";
    displayArray(dataset, size);

    std::cout << "\nInsertionsort: \n";
    insertionSort(dataset, size);
    displayArray(dataset, size);

    return 0;
}
```

```
template <typename T>
void insertionSort(T arr[], int size) {
    int k = 1, j;
    T temp;

    while (k < size) {
        temp = arr[k];
        j = k - 1;
        while (j >= 0 && temp <= arr[j]) {
            arr[j + 1] = arr[j];
            j--;
        }
        arr[j + 1] = temp;
        k++;
    }
}
```

OUTPUT

```
Unsorted Dataset1:
53 3 8 36 69 77 65 50 64 15 74 46 84 60 55 74 70 60 77 38 13 87 23 58 65 6 98 89 58 16 91 28 22 22 5 70 68 71 35 26 74 1
9 83 46 46 78 31 7 0 16 4 64 27 54 77 80 89 80 17 54 88 61 88 95 23 74 15 66 80 94 90 47 67 97 42 98 94 24 95 13 38 89 2
4 61 94 79 60 20 67 92 63 63 97 73 53 75 80 81 66 69
Insertionsort:
0 3 4 5 6 7 8 13 13 15 15 16 16 17 19 20 22 22 23 23 24 24 26 27 28 31 35 36 38 38 42 46 46 46 47 50 53 53 54 54 55 58 5
8 60 60 60 61 61 63 63 64 64 65 65 66 66 67 67 68 69 69 70 70 71 73 74 74 74 74 75 77 77 77 78 79 80 80 80 80 81 83 84 8
7 88 88 89 89 89 90 91 92 94 94 94 95 95 97 97 98 98
Process finished with exit code 0
```

OBSERVATION

After generating a new set of random values, I then used the insertion sort function. Insertion sorting has a time complexity of O(n^2) because the outer loop checks for the next number in the unsorted region to be put in the sorted region and the inner loop is the one responsible for checking and putting the number in the right place of the sorted region.

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Copy and paste the questions from the lab and include your answers after.

Candidate 1	Bo Dalton Capistrano
Candidate 2	Cornelius Raymon Agustín
Candidate 3	Deja Jayla Bañaga
Candidate 4	Lalla Brielle Yabut
Candidate 5	Franklin Relano Castro

Problem: Generate an array A[0…100] of unsorted elements, wherein the values in the array are indicative of a vote to a candidate. This means that the values in your array must only range from 1 to 5. Using sorting and searching techniques, develop an algorithm that will count the votes and indicate the winning candidate.

NOTE: The sorting techniques you have the option of using in this activity can be either bubble, selection, or insertion sort. Justify why you chose to use this sorting algorithm.

PSEUDOCODE OF ALGORITHM

Sorting Algorithm Pseudocode

```
ALGORITHM InsertionSort(arr, size)
   FOR k = 1 TO size - 1
      temp = arr[k]
      j = k - 1
      WHILE j >= 0 AND temp <= arr[j]
         arr[j + 1] = arr[j]
         j = j - 1
      END WHILE
      arr[j + 1] = temp
   END FOR
END ALGORITHM
```

Searching Algorithm Pseudocode

```
ALGORITHM LinearSearchCount(data, arrSize, target)
   counter = 0
   FOR i = 0 TO arrSize - 1
      IF data[i] == target THEN
         counter = counter + 1
      END IF
   END FOR
   RETURN counter
END ALGORITHM
```

SCREENSHOT OF ALGORITHM CODE

Sorting Algorithm (Insertion Sort)

```
template <typename T>
void insertionSort(T arr[], int size) {
    int k = 1, j;
    T temp;

    while (k < size) {
        temp = arr[k];
        j = k - 1;
        while (j >= 0 && temp <= arr[j]) {
            arr[j + 1] = arr[j];
            j--;
        }
        arr[j + 1] = temp;
        k++;
    }
}
```

Searching Algorithm (Linear Search)

```
template <typename T>
int linearSearchArr(T data[], int arrSize, T datafind) {
    int i = 0;
    int counter = 0;
    while (i < arrSize) {
        if (data[i] == datafind) {
            counter++;
        }
        i++;
    }
    return counter;
}
```

OUTPUT TESTING

OUTPUT CONSOLE SHOWING SORTED ARRAY

```
Unsorted Votes
3 1 4 3 1 4 3 2 3 5 4 2 5 5 2 4 5 2 5 4 5 4 3 5 2 3 1 2 4 4 1 4 5 2 2 1 5 1 2 5 5 4 3 1 5 5 2 1 2 2 4 2 2 4 5 5 2 5 1 1
1 1 2 3 1 4 3 3 5 2 1 3 3 1 1 5 4 5 1 3 4 2 5 5 2 1 4 2 4 1 3 2 3 1 4 4 5 3 3 1
Sorted Votes
1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 4
4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5
```

MANUAL COUNT

Manual Count

1 = 𝍸 - 𝍸 - 𝍸 - 𝍸 | - | 21

2 = 𝍸 - 𝍸 - 𝍸 - 𝍸 - | 21

3 = 𝍸 - 𝍸 - 𝍸 - || 17

4 = 𝍸 - 𝍸 - 𝍸 - |||| 19

5 = 𝍸 - 𝍸 - 𝍸 - 𝍸 - || 22

COUNT RESULT OF ALGORITHM

```
-----Votecount-----
Candidate 1, Bo Dalton Capistrano = 21
Candidate 2, Cornelius Raymon Agustin = 21
Candidate 3, Deja Jayla Banaga = 17
Candidate 4, Lalla Brielle Yabut = 19
Candidate 5, Franklin Relano Castro = 22
Franklin Relano Castro is the winner!

Process finished with exit code 0
```

QUESTION: Was your developed vote counting algorithm effective? Why or why not?

"I think my developed vote counting algorithm is effective. It sorted all the votes with the insertion sort technique in ascending order for easier manual counting and ran the modified linear search to count all the respective votes. Lastly, it determined the highest vote count correctly."

CONCLUSION

In this activity, I learned 3 sorting techniques; Bubble, Selection, and Insertion. I also successfully understood their time complexity, O(n^2), and why it is that way. For the procedures, we coded the actual sorting techniques. The bubble sort, we utilized a nested for loop. The outer loop for each pass of the entire array and the inner loop for the swapping of each adjacent element. For the selection sort, we also used a nested for loop, the outer being responsible for swapping the smallest element to the current element in the sorted region and the inner loop for finding the smallest value in the unsorted region. We mixed things up a bit for the insertion sort by using a nested while loop instead of a nested for loop. The outer while loop walks through the whole array with k marking the start of the unsorted region and the inner loop is responsible for the back tracking of the sorted region to find where k might fit.

Lastly, the supplementary activity was challenging. I solved this activity in this way, firstly we generated an array with a max size of 100 with random quantities of 1's, 2's, 3's, 4's, and 5's. Then we'll sort it by using the insertion sort, because it is the fastest among the three because it doesn't make unnecessary movements. After sorting, this is when we'll implement the modified linear search whereas it'll count the quantity of data in that array. This then will be returned as an integer value and be put in an array that stores all the candidates votes. Lastly, we'll match this array of vote counts to our array of candidates. Now we just use a simple function to find the highest value in the vote counts array and return the index value. In conclusion, it'll print out the corresponding vote count and the candidate aligned with that index value, thus effectively determining the winner.

In terms of time complexity, the insertion sort is O(n^2), the count of votes is O(n), and lastly in determining the highest it is O(1).

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **Gemini**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **Explanation of Concepts**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **Explained and Visualized the Concept of Bubble, Selection, and Insertion Sorting.**

COMPUTER ENGINEERING

QUEZON CITY

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.